

TD Séance 3 : L'IA qui Cherche

Exercices Progressifs — Algorithmes de Recherche

Feuille d'énoncés

Dr. El Hadji Bassirou TOURÉ

Département de Mathématiques et Informatique

Faculté des Sciences et Techniques

Université Cheikh Anta Diop de Dakar

M1 IABD — Semestre 1

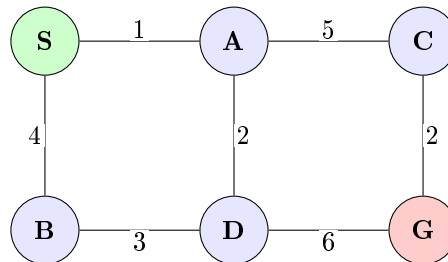
Organisation de ce TD

Ce TD est conçu pour une **compréhension progressive** des algorithmes de recherche :

- **Niveau 1** (Ex. 1–2) : Formulation de problèmes et vocabulaire
- **Niveau 2** (Ex. 3–5) : Exécution manuelle de BFS, DFS, UCS
- **Niveau 3** (Ex. 6–8) : Heuristiques, Greedy et A*
- **Niveau 4** (Ex. 9–10) : Comparaison, analyse et conception
- **Niveau 5** (Ex. 11–12) : Transfert sur nouveau graphe, vrai/faux théorique
- **Niveau 6** (Ex. 13–14) : Recherche locale, problème de synthèse

Conseil : Pour chaque algorithme, reproduisez le tableau d'exécution étape par étape.

Tous les exercices utilisent le graphe suivant :



Graphe de référence. S = départ (vert), G = but (rouge). Les nombres = coûts des arêtes.
Voisins : S : {A, B} A : {S, C, D} B : {S, D} C : {A, G} D : {A, B, G} G : {C, D}

Niveau 1 — Formulation de Problèmes

Exercice 1 : Formaliser un problème de recherche

Énoncé

En utilisant le graphe de référence :

1. Identifiez les cinq composants du problème de recherche : (S, s_0, G, A, c) .
2. Listez **tous** les chemins possibles de S à G (sans passer deux fois par le même nœud).
3. Pour chaque chemin, calculez le coût total.
4. Identifiez la solution optimale.

Votre réponse :

Exercice 2 : Vocabulaire fondamental

Énoncé

1. Qu'est-ce que la **frontière** (fringe) ? Quel est son contenu initial ?
2. Qu'est-ce qu'**expanser** (expand) un nœud ?
3. Quelle est la différence entre un nœud **généré** et un nœud **exploré** ?
4. Si on expande le nœud S, quels nœuds sont ajoutés à la frontière ?

5. Si on expande ensuite le nœud A, quels nœuds sont ajoutés ? Faut-il ajouter S ?

Votre réponse :

Niveau 2 — Recherche Non Informée (BFS, DFS, UCS)

Méthode pour tous les exercices de ce niveau

Pour chaque algorithme, remplissez un **tableau d'exécution** au format suivant (identique au cours) :

Ét.	Explore	Frontière après expansion	Parents mis à jour
0	–	[nœud initial]	–

Vocabulaire (cf. cours §3.3) :

- **Explorer** un nœud = le retirer de la frontière et générer ses successeurs
- **Frontière** = nœuds découverts mais pas encore explorés
- **Parent** : quand on ajoute n' via n , on note $n' \leftarrow n$

Rappel — Structures de données :

- **BFS** : File FIFO $\Rightarrow$ on explore le **plus ancien** (premier entré)
- **DFS** : Pile LIFO $\Rightarrow$ on explore le **plus récent** (dernier entré)
- **UCS** : File de priorité $\Rightarrow$ on explore celui de **plus petit** $g(n)$

Reconstruction du chemin : après avoir atteint le but, remontez les parents :

$$\text{But} \xleftarrow{\text{parent}} \dots \xleftarrow{\text{parent}} \text{Départ}$$

Exercice 3 : Recherche en Largeur (BFS)

Énoncé

Exécutez BFS sur le graphe de référence, de S vers G.

Convention : Successeurs en **ordre alphabétique**. Détection des états visités activée.

1. Remplissez le tableau d'exécution étape par étape.
2. Quel chemin BFS trouve-t-il ?
3. Quel est le coût de ce chemin ?
4. Ce chemin est-il optimal ? Pourquoi ?
5. Combien de nœuds BFS a-t-il explorés ?

Tableau d'exécution BFS (file FIFO) :

Ét.	Explore	Frontière après expansion	Parents mis à jour
0	–		–
1			
2			
3			
4			
5			
6			

Reconstruction du chemin : $G \xleftarrow{\text{parent}} \text{---} \xleftarrow{\text{parent}} \text{---} \xleftarrow{\text{parent}} S$
Chemin : _____ Coût : _____ Optimal ? _____ Nœuds
explorés : _____

Exercice 4 : Recherche en Profondeur (DFS)**Énoncé**

Exécutez DFS sur le même graphe, de S vers G.

Convention : Successeurs en **ordre alphabétique** sur la pile (le premier alphabétiquement au sommet). Détection des états visités activée.

1. Remplissez le tableau d'exécution.
2. Quel chemin DFS trouve-t-il ? Quel est son coût ?
3. Comparez avec BFS : mêmes résultats ?

Tableau d'exécution DFS (pile LIFO, sommet = élément en gras) :

Ét.	Explore	Pile après expansion	Parents mis à jour
0	—		—
1			
2			
3			
4			

Reconstruction : G $\xleftarrow{\text{parent}}$ _____ $\xleftarrow{\text{parent}}$ _____ $\xleftarrow{\text{parent}}$ S

Chemin : _____ Coût : _____

Comparaison BFS vs DFS :

	BFS	DFS
Chemin		
Coût		
Nœuds explorés		

Votre réponse :

Exercice 5 : Recherche à Coût Uniforme (UCS)**Énoncé**

Exécutez UCS sur le même graphe, de S vers G.

Convention : Détection des états visités activée. En cas d'égalité de $g(n)$, ordre alphabétique.

1. Remplissez le tableau d'exécution en indiquant $g(n)$ pour chaque nœud.
2. Quel chemin UCS trouve-t-il ?
3. Pourquoi peut-on garantir que ce chemin est optimal ?
4. À quelle étape UCS explore-t-il B ? Pourquoi pas avant ?

Tableau d'exécution UCS (priorité par $g(n)$ croissant) :

Ét.	Explore	g	Frontière (par g croissant)	Parent
0	—	—		—
1				
2				
3				
4				
5				
6				

Reconstruction : G $\xleftarrow{\text{parent}}$ _____ $\xleftarrow{\text{parent}}$ _____ $\xleftarrow{\text{parent}}$ S

Chemin : _____ Coût : _____ Optimal garanti ? _____

Votre réponse :

Niveau 3 — Recherche Informée (Heuristiques, Greedy, A*)

Exercice 6 : Comprendre les heuristiques

Énoncé

On définit les heuristiques suivantes (estimations du coût vers G) :

Nœud	S	A	B	C	D	G
$h_1(n)$	7	6	8	2	5	0
$h_2(n)$	10	8	12	1	7	0
$h_3(n)$	5	4	6	1	3	0

1. Rappelez la définition d'une heuristique **admissible**.
2. D'après l'exercice 1, quel est le vrai coût optimal $h^*(n)$ de chaque nœud vers G ?
3. Parmi h_1 , h_2 , h_3 , lesquelles sont admissibles ? Justifiez.
4. Parmi les admissibles, laquelle est la **plus informative** ? Pourquoi ?

Calcul des vrais coûts $h^(n)$ et vérification d'admissibilité :*

	S	A	B	C	D	G	Admissible ?
$h^*(n)$							—
$h_1(n)$	7	6	8	2	5	0	
$h_2(n)$	10	8	12	1	7	0	
$h_3(n)$	5	4	6	1	3	0	

Votre réponse :

Exercice 7 : Recherche Gloutonne (Greedy)**Énoncé**

Exécutez la recherche gloutonne avec h_1 :

Nœud	S	A	B	C	D	G
$h_1(n)$	7	6	8	2	5	0

1. Remplissez le tableau d'exécution (priorité = $h(n)$ croissant).
2. Quel chemin Greedy trouve-t-il ? Coût ?
3. Ce chemin est-il optimal ?
4. Pourquoi Greedy n'explore-t-il pas B ?

Tableau d'exécution Greedy (priorité par $h(n)$ croissant) :

Ét.	Explore	h	Frontière (par h croissant)	Parent
0	—	—		—
1				
2				
3				
4				

Reconstruction : G $\xleftarrow{\text{parent}}$ _____ $\xleftarrow{\text{parent}}$ _____ $\xleftarrow{\text{parent}}$ S

Chemin : _____ Coût : _____ Optimal ? _____

Votre réponse :

Exercice 8 : L'algorithme A***Énoncé**

Exécutez A* avec h_1 sur le graphe de référence.

Nœud	S	A	B	C	D	G
$h_1(n)$	7	6	8	2	5	0

1. Remplissez le tableau avec $g(n)$, $h(n)$, et $f(n) = g(n) + h(n)$.
2. Quel chemin A* trouve-t-il ?
3. Comparez le nombre de nœuds explorés par A*, UCS, et Greedy.
4. Que se passerait-il avec h_2 (non admissible) ?

Tableau d'exécution A* (priorité par $f(n) = g(n) + h(n)$ croissant) :

Ét.	Explore	g	h	f	Frontière (par f)	Parent
0	–					–
1						
2						
3						
4						
5						

Reconstruction : G $\xleftarrow{\text{parent}}$ _____ $\xleftarrow{\text{parent}}$ _____ $\xleftarrow{\text{parent}}$ S

Chemin : _____ Coût : _____

Comparaison finale de tous les algorithmes :

	BFS	DFS	UCS	Greedy	A*
Nœuds explorés					
Chemin					
Coût					
Optimal garanti ?					

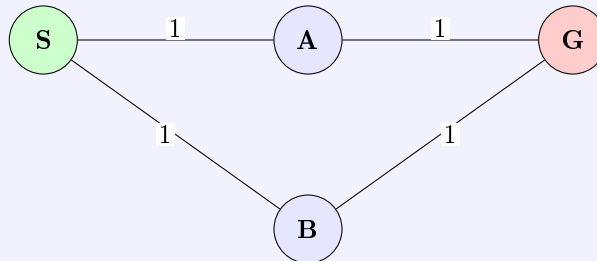
Votre réponse :

Niveau 4 — Comparaison, Analyse et Conception

Exercice 9 : Quand BFS bat A* (et inversement)

Énoncé

Considérez le graphe suivant :



1. Tous les coûts sont égaux à 1. Exécutez BFS et UCS. Trouvent-ils le même résultat ?
2. Maintenant, changez le coût de $A \rightarrow G$ à 100. Exécutez BFS et UCS. Que se passe-t-il ?
3. Conclusion : quand BFS suffit-il ? Quand faut-il UCS ou A* ?

Exercice 10 : Concevoir une heuristique par relaxation

Énoncé

Le **taquin** (8-puzzle) :

État actuel				État but		
7	2	4	$\Rightarrow$		1	2
5		6		3	4	5
8	3	1		6	7	8

1. Formulez comme problème de recherche (S, s_0, G, A, c) .
2. h_1 (**tuiles mal placées**) : comptez les tuiles hors position. Calculez h_1 .
3. h_2 (**distance de Manhattan**) : pour chaque tuile, $|x_{act} - x_{but}| + |y_{act} - y_{but}|$. Calculez h_2 .
4. Montrez que h_1 et h_2 sont admissibles par **relaxation**.
5. Laquelle est meilleure ? Pourquoi ?

h_1 — *Tuiles mal placées* :

Tuile	Position actuelle	Position but	Mal placée ?
1			
2			
3			
4			
5			
6			
7			
8			

$h_1 =$ _____

h_2 — *Distance de Manhattan* :

Tuile	Pos. actuelle (l,c)	Pos. but (l,c)	Manhattan
1			
2			
3			
4			
5			
6			
7			
8			
Total			

$h_2 =$ _____

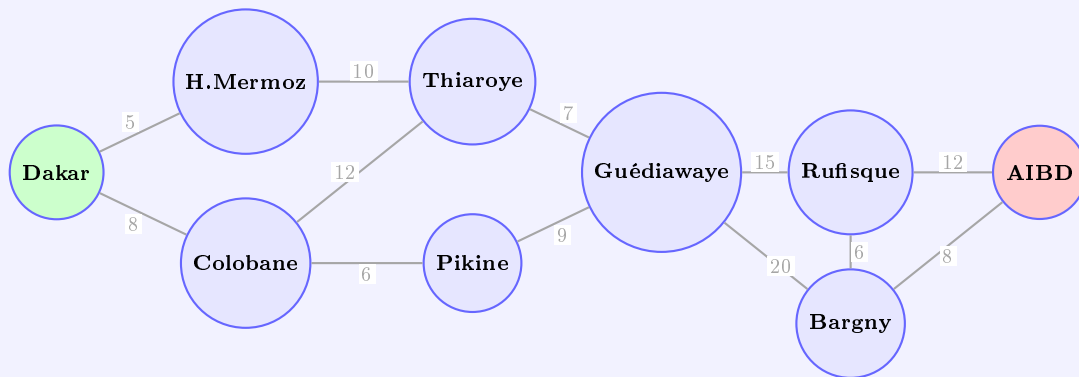
Votre réponse :

Niveau 5 — Transfert : Nouveau Graphe

Exercice 11 : Le réseau TER sénégalais

Énoncé

Le Train Express Régional (TER) relie Dakar à l'Aéroport AIBD via plusieurs gares. Considérons le graphe simplifié suivant (les coûts représentent les temps de trajet en minutes) :



Heuristiques $h(n)$ (temps estimé à vol d'oiseau vers AIBD, en minutes) :

	Dakar	H.Merm.	Colobane	Thiaroye	Pikine	Guédiaw.	Rufisque	Bargny	AIBD
h	35	30	32	22	24	18	10	7	0

1. Formulez ce problème : (S, s_0, G, A, c) , b , d optimal.
2. Exécutez **UCS** (Dakar $\rightarrow$ AIBD). Donnez le tableau complet.
3. Exécutez **A*** (Dakar $\rightarrow$ AIBD). Donnez le tableau complet.
4. Combien de nœuds chaque algorithme explore-t-il ? Quel est le gain apporté par l'heuristique ?
5. Vérifiez l'admissibilité de h : pour chaque nœud, est-ce que $h(n) \leq h^*(n)$?

Tableau UCS :

Ét.	Explore	g	Frontière (par g croissant)	Parent
0	–	–		–
1				
2				
3				
4				
5				
6				
7				
8				

Tableau A^* :

Ét.	Explore	g	h	f	Frontière (par f)	Parent
0	–					–
1						
2						
3						
4						
5						
6						
7						

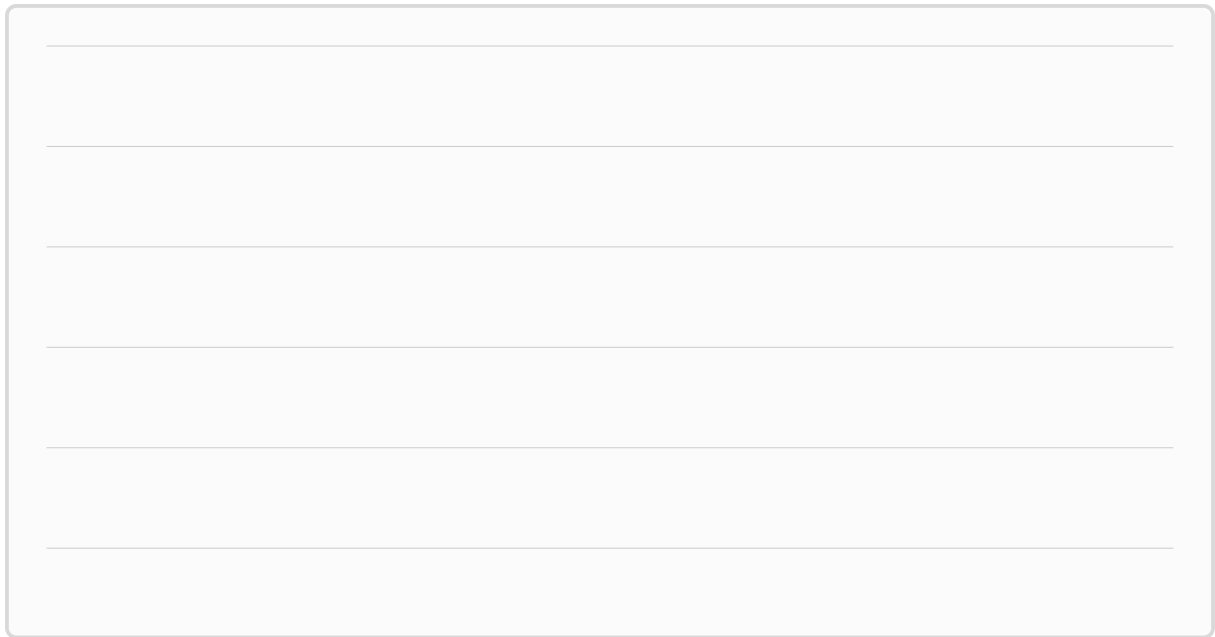
Votre réponse :

Exercice 12 : Vrai ou Faux — Propriétés des algorithmes**Énoncé**

Pour chaque affirmation, indiquez si elle est **vraie** ou **fausse**, et **justifiez** par un argument ou un contre-exemple.

1. « Si BFS trouve un chemin de coût 10, alors il n'existe pas de chemin de moins de 3 étapes. »
2. « DFS utilise moins de mémoire que BFS dans **tous** les cas. »
3. « Si $h(n) = 0$ pour tout n , alors A* se comporte exactement comme UCS. »
4. « Si l'heuristique h surestime le coût pour un seul nœud, A* peut manquer le chemin optimal. »
5. « UCS et Dijkstra trouvent toujours le même chemin optimal. »
6. « La distance de Manhattan est une heuristique admissible pour la navigation routière (comme dans notre fil rouge). »
7. « Si h_1 domine h_2 (toutes deux admissibles), alors A* avec h_1 explore toujours **strictement** moins de nœuds que A* avec h_2 . »
8. « Le recuit simulé peut trouver l'optimum global, mais pas hill climbing. »

Votre réponse :



A large rectangular box with a light gray background and a thin gray border, containing six horizontal lines for writing.

Niveau 6 — Pour Aller Plus Loin

Exercice 13 : Recherche locale — Le problème des 4 reines

Énoncé

Placer 4 reines sur un échiquier 4×4 tel qu’aucune paire ne s’attaque (pas sur la même ligne, colonne, ou diagonale).

Représentation : Un état est un vecteur (c_1, c_2, c_3, c_4) où c_i est la colonne de la reine de la ligne i . Chaque reine est sur une ligne distincte.

Fonction objectif : h = nombre de paires de reines en conflit (à minimiser, objectif : $h = 0$).

Voisinage : déplacer une reine dans sa ligne (changer c_i pour une valeur différente).

1. **État initial.** Considérez l’état $(1, 3, 2, 4)$.
 - (a) Dessinez l’échiquier correspondant (Q en ligne i , colonne c_i).
 - (b) Identifiez toutes les paires en conflit. Calculez h .
2. **Un pas de hill climbing.** Énumérez les 12 voisins (4 reines $\times$ 3 colonnes alternatives). Pour chacun, calculez h . Quel voisin hill climbing choisit-il ?
3. **Itérez :** à partir du voisin choisi, effectuez un deuxième pas de hill climbing. Avez-vous atteint $h = 0$? Si non, continuez.
4. **Recuit simulé.** Supposons qu’à un état avec $h = 1$, le recuit simulé propose un voisin avec $h = 2$ (dégradation $\Delta h = +1$), et que la température $T = 2$. Quelle est la probabilité d’accepter ce mouvement ? ($P = e^{-\Delta h/T}$). Quel est l’intérêt de cette acceptation ?

Échiquier 4×4 pour l’état $(1, 3, 2, 4)$:

Votre réponse :

Exercice 14 : Problème de synthèse — Livraison à Dakar**Énoncé**

Amadou gère un service de livraison de colis à Dakar. Il doit trouver le trajet le plus rapide entre le Point E (Marché Sandaga) et le Point F (Université UCAD). Le réseau routier simplifié est le suivant (coûts en minutes) :

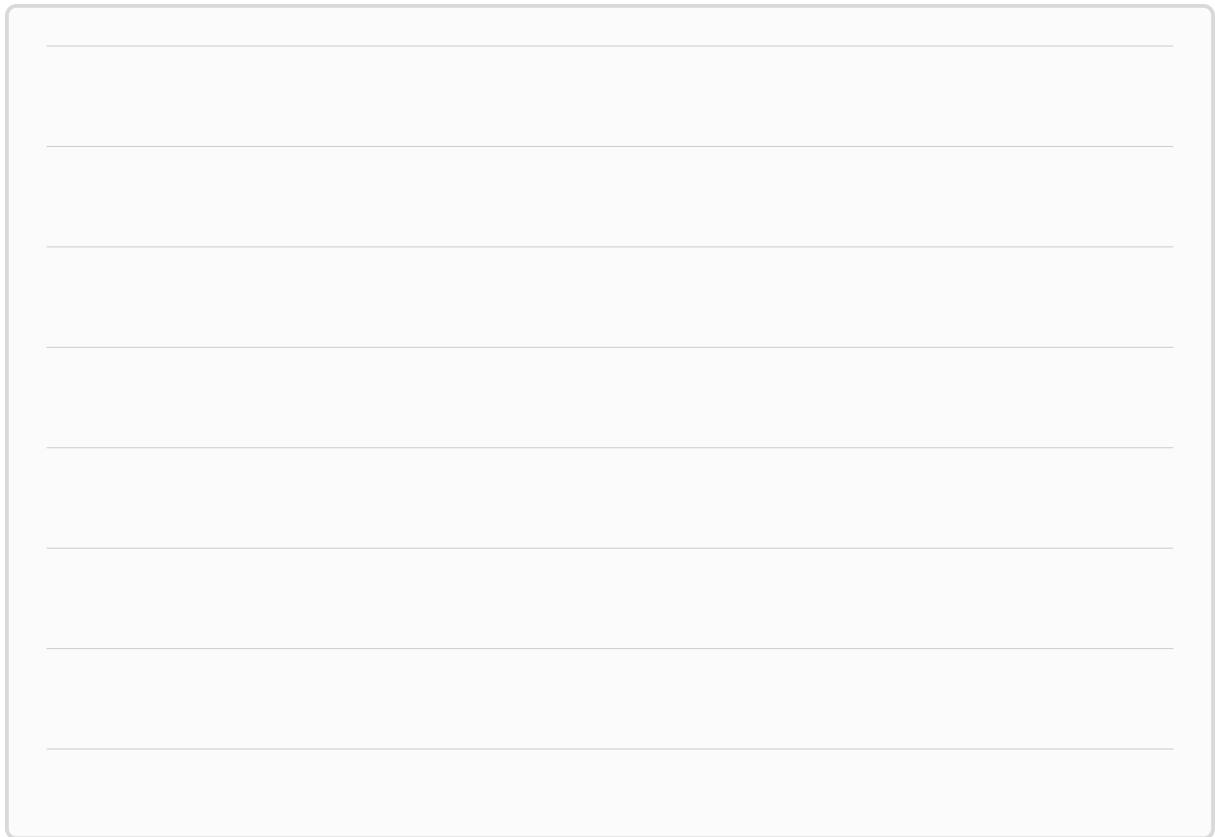
$$\begin{array}{lll} E \rightarrow A : 3 & A \rightarrow C : 4 & C \rightarrow F : 5 \\ E \rightarrow B : 5 & A \rightarrow D : 7 & D \rightarrow F : 2 \\ B \rightarrow C : 2 & B \rightarrow D : 6 & C \rightarrow D : 1 \end{array}$$

Attention : toutes les arêtes sont **orientées** (sens unique). Le graphe est dirigé.
Heuristiques proposées :

	E	A	B	C	D	F
h_α	8	6	5	4	2	0
h_β	12	9	7	6	2	0

1. Dessinez le graphe orienté. Trouvez le chemin optimal (par inspection ou UCS).
2. h_α **est-elle admissible** ? Calculez $h^*(n)$ pour chaque nœud et vérifiez.
3. h_β **est-elle admissible** ? Même vérification. Si non, identifiez les nœuds problématiques.
4. Exécutez **A*** avec h_α . Trouvez-vous le chemin optimal ?
5. Exécutez **A*** avec h_β . Trouvez-vous le chemin optimal ? Expliquez.
6. Construisez une heuristique h_γ qui **domine** h_α tout en restant admissible. (*Indice : utilisez $h_\gamma(n) = \max(h_\alpha(n), \dots)$*)

Votre réponse :



A large rectangular box with a light gray background and a thin gray border, containing eight horizontal lines for writing.

Aide-mémoire

	BFS	DFS	UCS	Greedy	A*
Frontière	File FIFO	Pile LIFO	Priorité $g(n)$	Priorité $h(n)$	Priorité $f(n)$
Complet ?	Oui	Non	Oui	Non	Oui
Optimal ?	Non*	Non	Oui	Non	Oui**
Temps	$O(b^d)$	$O(b^m)$	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$	$O(b^m)$	$\leq$ UCS
Espace	$O(b^d)$	$O(bm)$	Comme temps	$O(b^m)$	Comme temps
Informé ?	Non	Non	Non	Oui	Oui

* BFS optimal si coûts égaux. ** A* optimal si h admissible.

b = branchement, d = profondeur solution, m = profondeur max, C^* = coût optimal, ε = coût min.